Informe Oficial de Entregables — Prueba Técnica de Sincronización Asíncrona de Productos (Grupo ASD)

> **Propósito de este documento:**
> Este documento organiza punto por punto los **7 Entregables oficiales** solicitados en la prueba técnica de Grupo ASD, incluyendo la explicación de ambas aplicaciones en **Java 25 LTS (25.0.5)** y **Spring Boot 3.5.16**, su configuración externa, manual de instalación y ejecución, diagrama de arquitectura y flujo de mensajes, evidencias completas de los **9 escenarios de prueba** (con **33 pruebas automatizadas: 15 unitarias, 15 de integración contra PostgreSQL 16 y ActiveMQ real, y 3 de flujo completo E2E** más **14 capturas reales**), explicación de los mecanismos de productos pendientes, prevención de duplicados y manejo de errores, y el despliegue integral con **Docker Compose**.

Índice de Entregables Cubiertos en este Documento

#	Entregable Solicitado en la Prueba Técnica	Sección en este Documento / Archivo en el Proyecto	Estado
1	**Código fuente de ambas aplicaciones**	**Entregable 1 (Sección 1)** — Módulos `product-publisher-service` y `product-consumer-service`	**ENTREGADO**
2	**Configuración necesaria para ejecutar los componentes**	**Entregable 2 (Sección 2)** — Archivos `application.yml`, `.env.example` e `init-schemas.sql`	**ENTREGADO**
3	**Archivo README con instrucciones de instalación, configuración y ejecución**	**Entregable 3 (Sección 3)** y archivo `README.md` en la raíz del repositorio	**ENTREGADO**
4	**Diagrama de arquitectura y flujo de mensajes**	**Entregable 4 (Sección 4)** — Imagen de arquitectura y flujo detallado de los 14 pasos	**ENTREGADO**

#	Entregable Solicitado en la Prueba Técnica	Sección en este Documento / Archivo en el Proyecto	Estado
5	**Evidencias de las pruebas realizadas (9 escenarios + pruebas unitarias, integración y E2E)**	**Entregable 5 (Sección 5)** — 33 tests automatizados + 14 capturas en Postman, pgAdmin 4 y ActiveMQ	**ENTREGADO**
6	**Breve explicación del mecanismo de identificación de productos pendientes, prevención de duplicados y manejo de errores**	**Entregable 6 (Sección 6)** — Explicación técnica detallada de los tres mecanismos	**ENTREGADO**
7	**Ejecución del entorno completo mediante Docker Compose (Valor Agregado)**	**Entregable 7 (Sección 7)** y archivo `docker-compose.yml` en la raíz	**ENTREGADO**

ENTREGABLE 1: Código Fuente de Ambas Aplicaciones

La solución está construida como un proyecto Maven multi-módulo compuesto por **dos microservicios Spring Boot independientes** bajo principios de Clean Code, separación de capas y principios SOLID:

- **Aplicación 1 — `product-publisher-service` (Puerto `8080` — Esquema `grupoasd_dev`):** Expone la API REST para registrar y consultar productos en el sistema de origen, ejecuta periódicamente el Job programado (`@Scheduled`) que identifica productos pendientes (`publicado = false`), publica un mensaje JSON por producto en ActiveMQ y escucha la cola de confirmación para actualizar `notificado = true`.
- **Aplicación 2 — `product-consumer-service` (Puerto `8081` — Esquema `grupoasd_prod`):** Permanece suscrita a la cola de ActiveMQ (`@JmsListener`), valida cada mensaje recibido, verifica que el código no exista previamente en el esquema de producción (prevención de duplicados), persiste el producto en `grupoasd_prod.producto` y envía la confirmación de retorno al Componente 1.

1.1. Estructura de Paquetes y Clases de Ambas Aplicaciones

```
grupoasd-product-sync/

├── pom.xml                                  <-- POM Padre Maven (Java 25 LTS + Spring Boot
3.5.16)
```

```
│
├── product-publisher-service/                <-- APLICACIÓN 1 (Puerto 8080 - Esquema: grupoasd_dev)
│   ├── Dockerfile
│   ├── pom.xml
│   ├── src/main/java/com/grupoasd/publisher/
│   │   ├── ProductPublisherApplication.java             (Clase principal con @SpringBootApplication)
│   │   ├── config/JmsSchedulerConfig.java               (Configuración @EnableJms, @EnableScheduling y Jackson JSON)
│   │   ├── controller/ProductoController.java           (Controlador REST: POST y GET /api/v1/productos)
│   │   ├── service/ProductoService.java                 (Contrato de la lógica de negocio en origen)
│   │   ├── service/impl/ProductoServiceImpl.java        (Implementación transaccional de registro y estados)
│   │   ├── scheduler/ProductoPublicacionJob.java        (Job @Scheduled que identifica pendientes y publica)
│   │   ├── messaging/ProductoMessageProducer.java       (Productor JMS hacia grupoasd.productos.registro.queue)
│   │   ├── messaging/ProductoConfirmacionListener.java  (Listener JMS de confirmaciones desde grupoasd_prod)
│   │   ├── repository/ProductoRepository.java           (Repositorio Spring Data JPA para grupoasd_dev.producto)
│   │   ├── domain/entity/Producto.java                  (Entidad JPA con campos de negocio y control de publicación)
│   │   ├── dto/                                         (ProductoRequestDTO, ProductoResponseDTO, ProductoMensajeDTO)
│   │   ├── mapper/ProductoMapper.java                   (Conversión desacoplada entre Entidad y DTOs)
│   │   └── exception/GlobalExceptionHandler.java        (Manejo centralizado de errores HTTP 400, 404, 409 y 500)
│   └── src/test/java/com/grupoasd/publisher/
│       ├── ProductoServiceUnitTest.java                 (9 Pruebas Unitarias puras con JUnit 5 + Mockito)
│       ├── ProductPublisherIntegrationTest.java         (9 Pruebas de Integración con @SpringBootTest + PostgreSQL + ActiveMQ)
│       └── FlujoCompletoSincronizacionE2ETest.java      (3 Pruebas de Flujo Completo E2E entre ambas aplicaciones)
│
└── product-consumer-service/                 <-- APLICACIÓN 2 (Puerto 8081 - Esquema: grupoasd_prod)
```

```
    ├── Dockerfile
    ├── pom.xml
    ├── src/main/java/com/grupoasd/consumer/
    │   ├── ProductConsumerApplication.java              (Clase principal con
@SpringBootApplication)
    │   ├── config/JmsConsumerConfig.java                (Sesión SESSION_TRANSACTED,
RedeliveryPolicy y DLQ)
    │   ├── messaging/ProductoConsumerListener.java      (Consumidor @JmsListener desde
ActiveMQ)
    │   ├── messaging/NotificacionJmsPublisher.java      (Publicador JMS de confirmación hacia
Componente 1)
    │   ├── service/ProductoSincronizacionService.java   (Contrato de sincronización en
grupoasd_prod)
    │   ├── service/impl/ProductoSincronizacionServiceImpl.java (Persistencia idempotente sin
duplicados)
    │   ├── controller/ProductoDestinoController.java    (API REST de consulta en destino:
/api/v1/productos-destino)
    │   ├── repository/ProductoDestinoRepository.java    (Repositorio Spring Data JPA para
grupoasd_prod.producto)
    │   ├── domain/entity/ProductoDestino.java           (Entidad JPA mapeada a
grupoasd_prod.producto)
    │   └── dto/                                         (ProductoEventMessage y
ProductoNotificacionMessage)
    └── src/test/java/com/grupoasd/consumer/
        ├── ProductoSincronizacionServiceUnitTest.java   (6 Pruebas Unitarias puras con JUnit
5 + Mockito)
        └── ProductConsumerIntegrationTest.java          (6 Pruebas de Integración con
@SpringBootTest + PostgreSQL + ActiveMQ)
```

1.2. Modelo de Datos Implementado en Origen (`grupoasd_dev`) y Producción (`grupoasd_prod`)

Campo	Tipo en Java / SQL	En `grupoasd_dev.producto`	En `grupoasd_prod.producto`	Descripción
`id`	`Long` / `BIGSERIAL PK`	Sí	Sí	Identificador único autogenerado en cada esquema.

Campo	Tipo en Java / SQL	En `grupoasd_dev.producto`	En `grupoasd_prod.producto`	Descripción
`idOrigen` (`id_origen`)	`Long` / `BIGINT`	No	Sí	ID original del producto en `grupoasd_dev` para trazabilidad.
`codigo`	`String` / `VARCHAR(60)` `UNIQUE`	Sí	Sí	Código único del producto (normalizado en mayúsculas).
`nombre`	`String` / `VARCHAR(150)`	Sí	Sí	Nombre descriptivo del producto.
`descripcion`	`String` / `VARCHAR(500)`	Sí	Sí	Descripción detallada del producto.
`precio`	`BigDecimal` / `NUMERIC(19,2)`	Sí	Sí	Precio unitario del producto (validado `> 0`).
`fechaCreacion`	`LocalDateTime` / `TIMESTAMP`	Sí	Sí	Fecha y hora de creación del producto.
`publicado`	`boolean` / `BOOLEAN`	Sí (`false` inicial)	No	Pasa a `true` cuando el Job publica el mensaje en ActiveMQ.

Campo	Tipo en Java / SQL	En `grupoasd_dev.producto`	En `grupoasd_prod.producto`	Descripción
`notificado`	`boolean` / `BOOLEAN`	Sí (`false` inicial)	No	Pasa a `true` cuando el Componente 2 confirma persistencia en `grupoasd_prod`.
`fechaPublicacion`	`LocalDateTime` / `TIMESTAMP`	Sí	No	Fecha y hora exacta de envío a la cola ActiveMQ.
`fechaNotificacion`	`LocalDateTime` / `TIMESTAMP`	Sí	No	Fecha y hora exacta de confirmación por parte del Componente 2.
`intentosPublicacion`	`Integer` / `INTEGER`	Sí (`0` inicial)	No	Contador de intentos de publicación realizados por el Job.
`ultimoErrorPublicacion`	`String` / `VARCHAR(500)`	Sí	No	Detalle del último error registrado si falló la publicación en ActiveMQ.
`fechaSincronizacion`	`LocalDateTime` / `TIMESTAMP`	No	Sí	Fecha y hora exacta en que fue persistido en `grupoasd_prod`.

1.3. Versiones y Compatibilidad de Dependencias Documentadas

Tecnología / Dependencia	Versión	Justificación de Compatibilidad
Java (JDK)	`25 LTS (25.0.5)`	Versión LTS configurada en `pom.xml` (`<java.version>25</java.version>`) y en los Dockerfiles (`eclipse-temurin:25-jre-alpine`).
Spring Boot	`3.5.16`	Rama empresarial 3.5.x compatible nativamente con Java 25, Jakarta EE 10, Hibernate 6.6 y ActiveMQ 6.
Apache ActiveMQ Classic	`6.3.2 / 5.18.6`	Cliente `activemq-client-jakarta` compatible con el API `jakarta.jms.*` de Spring Boot 3.5.
PostgreSQL / JDBC Driver	`16-alpine / 42.7.14`	Soporte completo de múltiples esquemas (`grupoasd_dev` y `grupoasd_prod`) e índices compuestos.
Springdoc OpenAPI	`2.8.17`	Compatible con Spring Framework 6.2 / Spring Boot 3.5 para generación de Swagger UI.

ENTREGABLE 2: Configuración Necesaria para Ejecutar los Componentes

Toda la configuración de base de datos, esquemas, bróker ActiveMQ, nombres de colas, política de reintentos y periodicidad del Job se encuentra **externalizada** en archivos `application.yml` y variables de entorno, sin ningún valor ni credencial codificado directamente en las clases Java:

2.1. Configuración Externa del Componente 1 (`product-publisher-service/src/main/resources/application.yml`)

```
server:

  port: ${SERVER_PORT:8080}
```

```
spring:
  datasource:
    url:
${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/grupoasd_db?currentSchema=grupoasd_de
v}
    username: ${SPRING_DATASOURCE_USERNAME:grupoasd_user}
    password: ${SPRING_DATASOURCE_PASSWORD:grupoasd_pass}
  jpa:
    hibernate:
      ddl-auto: ${HIBERNATE_DDL_AUTO:update}
    properties:
      hibernate:
        default_schema: ${DB_SCHEMA_DEV:grupoasd_dev}
  activemq:
    broker-url: ${SPRING_ACTIVEMQ_BROKER_URL:tcp://localhost:61616}
    user: ${SPRING_ACTIVEMQ_USER:admin}
    password: ${SPRING_ACTIVEMQ_PASSWORD:admin}


app:
  messaging:
    queue:
      productos: ${QUEUE_PRODUCTOS_REGISTRO:grupoasd.productos.registro.queue}
      confirmacion: ${QUEUE_PRODUCTOS_CONFIRMACION:grupoasd.productos.notificacion.queue}
  scheduler:
    publicacion:
      fixed-delay-ms: ${JOB_CRON_INTERVAL_MS:10000}
      initial-delay-ms: ${JOB_INITIAL_DELAY_MS:5000}
      batch-size: ${JOB_BATCH_SIZE:50}
      reenvio-no-notificados-segundos: ${JOB_REENVIO_SEGUNDOS:20}
```

2.2. Configuración Externa del Componente 2 (`product-consumer-service/src/main/resources/application.yml`)

```
server:
```

```
  port: ${SERVER_PORT:8081}


spring:

  datasource:

    url:
${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/grupoasd_db?currentSchema=grupoasd_pr
od}

    username: ${SPRING_DATASOURCE_USERNAME:grupoasd_user}

    password: ${SPRING_DATASOURCE_PASSWORD:grupoasd_pass}

  jpa:

    hibernate:

      ddl-auto: ${HIBERNATE_DDL_AUTO:update}

    properties:

      hibernate:

        default_schema: ${DB_SCHEMA_PROD:grupoasd_prod}

  activemq:

    broker-url: ${SPRING_ACTIVEMQ_BROKER_URL:tcp://localhost:61616}

    user: ${SPRING_ACTIVEMQ_USER:admin}

    password: ${SPRING_ACTIVEMQ_PASSWORD:admin}


app:

  messaging:

    queue:

      productos: ${QUEUE_PRODUCTOS_REGISTRO:grupoasd.productos.registro.queue}

      confirmacion: ${QUEUE_PRODUCTOS_CONFIRMACION:grupoasd.productos.notificacion.queue}

    redelivery:

      max-redeliveries: ${JMS_MAX_REDELIVERIES:3}

      initial-delay-ms: ${JMS_INITIAL_REDELIVERY_DELAY_MS:2000}

      multiplier: ${JMS_BACKOFF_MULTIPLIER:2.0}

      use-exponential-backoff: true
```

2.3. Script de Creación Inicial de Esquemas (`docker/postgres/init-schemas.sql`)

```
CREATE SCHEMA IF NOT EXISTS grupoasd_dev AUTHORIZATION grupoasd_user;
```

```
CREATE SCHEMA IF NOT EXISTS grupoasd_prod AUTHORIZATION grupoasd_user;

GRANT ALL PRIVILEGES ON SCHEMA grupoasd_dev TO grupoasd_user;

GRANT ALL PRIVILEGES ON SCHEMA grupoasd_prod TO grupoasd_user;
```

ENTREGABLE 3: Instrucciones de Instalación, Configuración y Ejecución (Resumen del README)

El repositorio incluye el archivo `README.md` en la raíz con la guía paso a paso de instalación, configuración y ejecución:

3.1. Opción A: Ejecución Completa con Docker Compose (1 Solo Comando)

```
# 1. Situarse en la carpeta raíz del proyecto:

cd grupoasd-product-sync


# 2. Construir y levantar PostgreSQL 16, ActiveMQ y ambos microservicios:

docker compose up -d --build


# 3. Verificar que los 4 contenedores estén en estado Up / Healthy:

docker compose ps
```

3.2. Opción B: Ejecución en Modo Local con Maven y Java 25 LTS

```
# 1. Levantar únicamente la infraestructura base (PostgreSQL + ActiveMQ):

docker compose up -d postgres-db activemq-broker


# 2. Ejecutar las 20 pruebas automatizadas (Unitarias + Integración):

mvn clean test


# 3. Iniciar el Componente 1 (Puerto 8080 - Esquema grupoasd_dev):

mvn -pl product-publisher-service spring-boot:run


# 4. En otra terminal, iniciar el Componente 2 (Puerto 8081 - Esquema grupoasd_prod):

mvn -pl product-consumer-service spring-boot:run
```

3.3. Tabla de Accesos y Credenciales del Entorno

Servicio	URL / Dirección de Acceso	Credenciales / Detalles
Componente 1 — API REST (`grupoasd_dev`)	`http://localhost:8080/api/v1/productos`	Swagger UI: `http://localhost:8080/swagger-ui.html`
Componente 2 — API REST (`grupoasd_prod`)	`http://localhost:8081/api/v1/productos-destino`	Swagger UI: `http://localhost:8081/swagger-ui.html`
Consola Web Apache ActiveMQ	`http://localhost:8161/admin/queues.jsp`	Usuario: `admin` / Contraseña: `admin`
PostgreSQL 16 (pgAdmin 4 / DBeaver)	Host: `localhost` — Puerto: `5432`	BD: `grupoasd_db` | Usuario: `grupoasd_user` | Clave: `grupoasd_pass`

ENTREGABLE 4: Diagrama de Arquitectura y Diagrama de Flujo de Mensajes (Separados en Draw.io y PNG)

Para mayor claridad técnica, el diseño se entrega dividido en **dos diagramas independientes** (ambos disponibles también en formato editable de Draw.io en `docs/diagrama-arquitectura.drawio` y `docs/diagrama-flujo-mensajes.drawio`):

4.1. Diagrama 1: Arquitectura de Componentes e Infraestructura (Docker Compose)

Ilustra los cuatro contenedores que conforman la solución dentro de la red `grupoasd-sync-network` (`product-publisher-service` en el puerto `8080`, `activemq-broker` en los puertos `61616/8161`, `product-consumer-service` en el puerto `8081` y `postgres-db` en el puerto `5432`), sus capas internas y los dos esquemas aislados (`grupoasd_dev` y `grupoasd_prod`):

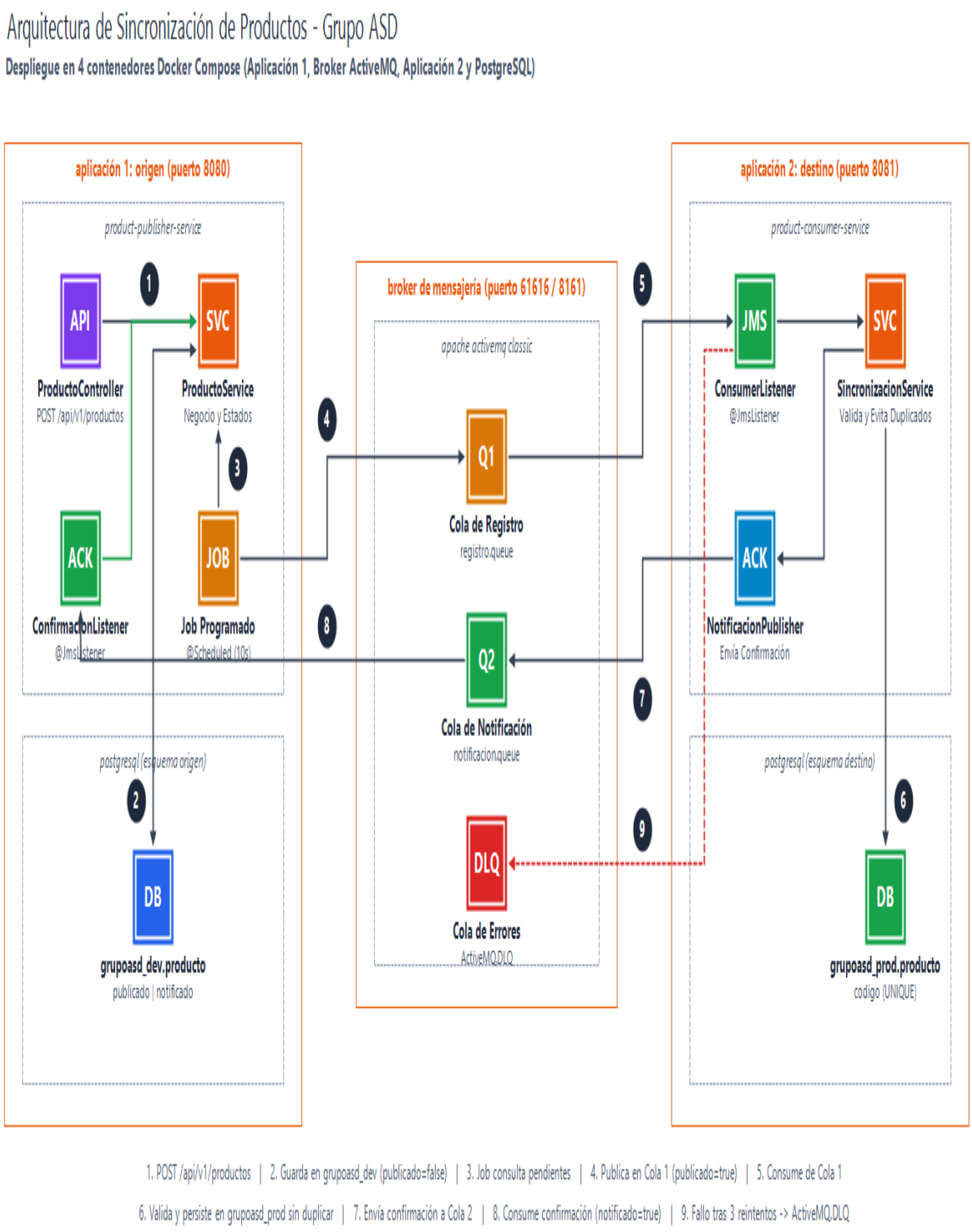


Figura 0A: Diagrama 1 — Arquitectura de Componentes e Infraestructura en Docker Compose (Archivo fuente: docs/diagrama-arquitectura.drawio).

4.2. Diagrama 2: Flujo de Mensajes y Transición de Estados (Pasos 1 al 10 + Manejo de Errores)

Ilustra la secuencia cronológica paso a paso del mensaje desde el registro inicial por API REST (`publicado = false, notificado = false`), la detección y publicación por el Job en ActiveMQ (`publicado = true, notificado = false`), el consumo transaccional con prevención de duplicados en `grupoasd_prod.producto`, el envío a `ActiveMQ.DLQ` ante fallos reiterados, y la confirmación de retorno que actualiza `notificado = true`:

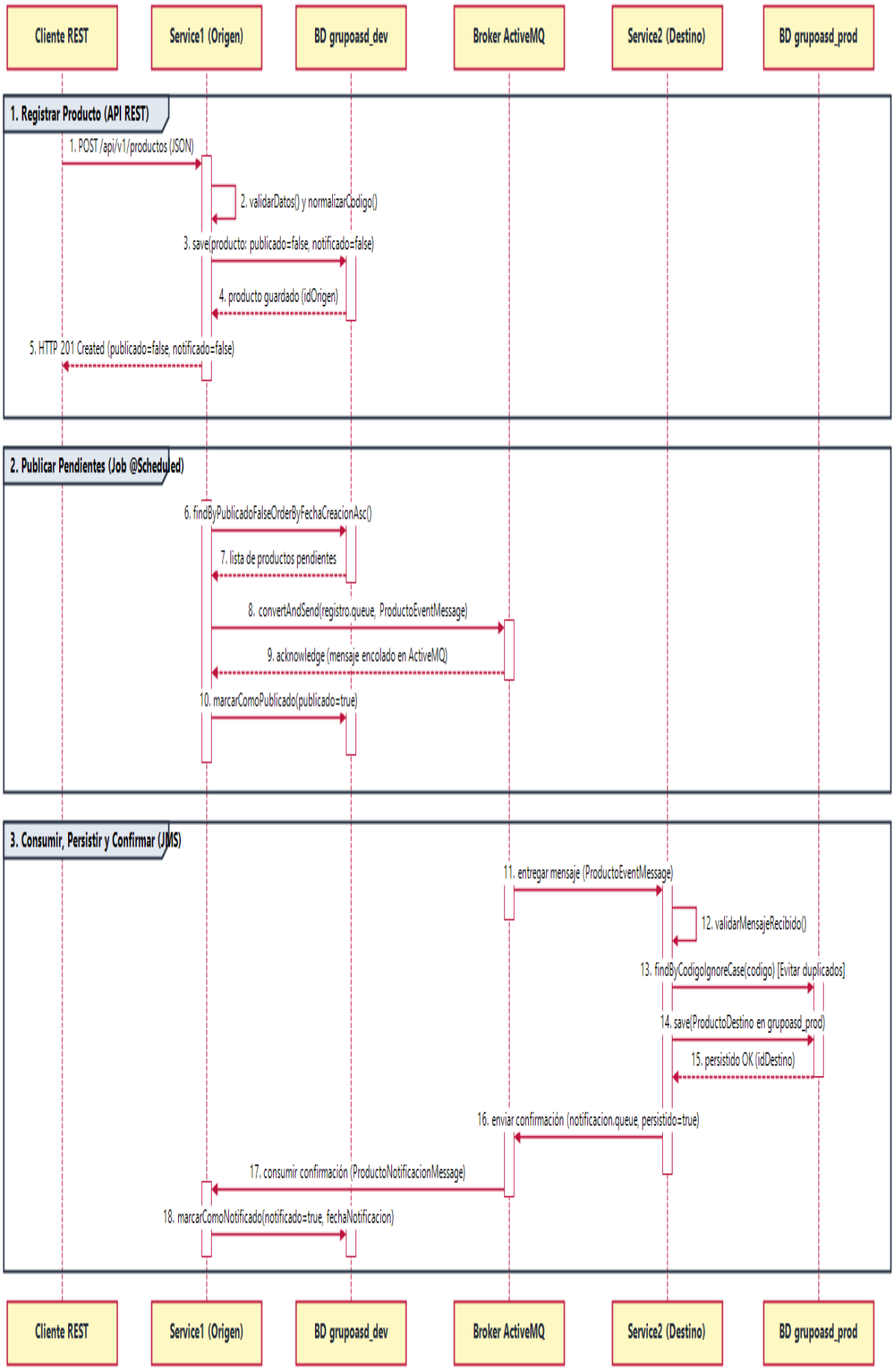

Cliente REST
Service1 (Origen)
BD grupoasd_dev
Broker ActiveMQ
Service2 (Destino)
BD grupoasd_prod
1. Registrar Producto (API REST)
1. POST /api/v1/productos (JSON)
2. validarDatos() y normalizarCodigo()
3. save(producto: publicado=false, notificado=false)
4. producto guardado (idOrigen)
5. HTTP 201 Created (publicado=false, notificado=false)
2. Publicar Pendientes (Job @Scheduled)
6. findByPublicadoFalseOrderByFechaCreacionAsc()
7. lista de productos pendientes
8. convertAndSend(registro.queue, ProductoEventMessage)
9. acknowledge (mensaje encolado en ActiveMQ)
10. marcarComoPublicado(publicado=true)
3. Consumir, Persistir y Confirmar (JMS)
11. entregar mensaje (ProductoEventMessage)
12. validarMensajeRecibido()
13. findByCodigoIgnoreCase(codigo) [Evitar duplicados]
14. save(ProductoDestino en grupoasd_prod)
15. persistido OK (idDestino)
16. enviar confirmación (notificacion.queue, persistido=true)
17. consumir confirmación (ProductoNotificacionMessage)
18. marcarComoNotificado(notificado=true, fechaNotificacion)
Cliente REST
Service1 (Origen)
BD grupoasd_dev
Broker ActiveMQ
Service2 (Destino)
BD grupoasd_prod

Figura 0B: Diagrama 2 — Flujo de Mensajes y Estados de Sincronización Paso a Paso (Archivo fuente: docs/diagrama-flujo-mensajes.drawio).

4.3. Descripción Paso a Paso del Flujo de Mensajes

1. **Registro REST (Pasos 1 y 2):** Un cliente invoca `POST /api/v1/productos` en el Componente 1 (puerto `8080`). `ProductoServiceImpl` valida los datos y la unicidad del código y guarda el registro en `grupoasd_dev.producto` con **`publicado = false`** y **`notificado = false`**.
2. **Detección por el Job y Publicación en ActiveMQ (Pasos 3, 4, 5 y 6):** Cada 10 segundos, `ProductoPublicacionJob` consulta los productos con `publicado = false`, construye un `ProductoMensajeDTO` (JSON) individual por cada producto, lo envía a la cola **`grupoasd.productos.registro.queue`** y actualiza el producto en origen a **`publicado = true`**.
3. **Consumo, Validación y Persistencia en Producción (Pasos 7, 8 y 9):** `ProductoMessageConsumer` en el Componente 2 (puerto `8081`) consume el mensaje automáticamente desde ActiveMQ en sesión transaccional, valida sus campos, verifica que el código no exista en `grupoasd_prod.producto` y lo persiste. Si un mensaje falla tras 3 reintentos exponenciales, se desvía a **`ActiveMQ.DLQ`**.
4. **Confirmación de Retorno al Origen (Pasos 10, 11, 12 y 13):** Una vez persistido en `grupoasd_prod.producto`, el Componente 2 publica un mensaje en **`grupoasd.productos.notificacion.queue`**. El Componente 1 lo recibe y actualiza **`notificado = true`** y `fecha_notificacion` en `grupoasd_dev.producto`.

ENTREGABLE 5: Evidencias de las Pruebas Realizadas (9 Escenarios + 33 Pruebas Automatizadas: 15 Unitarias, 15 de Integración y 3 Flujo Completo E2E)

5.1. Matriz de Verificación de los 9 Escenarios Mínimos Requeridos

#	Escenario Requerido en la Prueba	Prueba Unitaria (Mockito) + Integración (@SpringBootTest) + Flujo Completo E2E	Ev Do
1	**Registro exitoso de un producto mediante la API REST**	**Unitaria:** `ProductoServiceUnitTest` → `debeRegistrarProductoValidoConEstadoInicialPendiente` y `debeNormalizarCodigoEnMayusculasYSinEspaciosExtremos` **Integración:** `ProductPublisherIntegrationTest` → `debeRegistrarProductoExitosamentePorApiRest`	**Ev** (Po Cr pu no

#	Escenario Requerido en la Prueba	Prueba Unitaria (Mockito) + Integración (@SpringBootTest) + Flujo Completo E2E	Ev Do
2	**Rechazo de productos con información inválida o códigos duplicados**	**Unitaria:** `debeRechazarCodigoDuplicadoSinGuardar` y `debeRechazarDatosInvalidosEnDtoDeSolicitud` **Integración:** `debeRechazarProductoConCodigoDuplicado`, `debeRechazarProductoConInformacionInvalida` y `debeRechazarConMensajeClaroCuandoPrecioContieneTexto`	**Ev** Co **Ev** Ba Po
3	**Identificación de productos pendientes por parte del Job**	**Unitaria:** `jobDebeIdentificarProductosPendientesYProcesarlosEnOrden` **Integración:** `jobDebeIdentificarPendientesYPublicarEnActiveMQ`	**Ev** /p pu **Ev** de
4	**Publicación correcta de los mensajes en ActiveMQ**	**Unitaria:** `debeConstruirMensajeJmsCorrectamenteYMarcarPublicadoTrue`, `debeActualizarNotificadoATrueAlProcesarConfirmacionValida` y `debeIgnorarConfirmacionInvalidaOManejarConfirmacionRepetidaDeFormaIdempotente` **Integración:** `debeActualizarNotificadoATrueCuandoComponente2ConfirmaPersistencia` **E2E:** `FlujoCompletoSincronizacionE2ETest` (10 pasos completos)	**Ev** (pu no y **E** (Co Ac
5	**Consumo y persistencia de productos en `grupoasd_prod`**	**Unitaria:** `ProductoSincronizacionServiceUnitTest` → `debePersistirProductoNuevoYEnviarConfirmacionCorrecta` y `debeConstruirConfirmacionConTodosLosDatosDeTrazabilidad` **Integración:** `ProductConsumerIntegrationTest` → `debeConsumirPersistirYNotificarAlComponente1`	**Ev** (Po **Ev** Co **Ev** (pg
6	**Prevención de duplicados ante la recepción repetida de mensajes**	**Unitaria:** `debeDetectarProductoExistenteYEvitarDuplicadosAnteMensajesRepetidos` y `debeEvitarDuplicadosAnteCondicionDeCarreraConcurrente` **Integración y E2E:** `debePrevenirDuplicadosAnteRecepcionRepetidaDeMensajes` y `noDebeDuplicarEnDestinoCuandoSeReenviaElMismoMensajeEnFlujoCompleto`	**Ev** (Ac me pro du
7	**Recuperación ante fallos**	**Unitaria:** `debeMantenerProductoPendienteCuandoPublicacionEnActiveMqFalla` y `noDebeConfirmarProductoCuandoPersistenciaFallaYDebePropagarErrorParaReintento`	**Ev** (Po

#	Escenario Requerido en la Prueba	Prueba Unitaria (Mockito) + Integración (@SpringBootTest) + Flujo Completo E2E	Ev Do
	temporales de publicación o persistencia	**Integración y E2E:** `jobDebeGestionarFalloTemporalDePublicacionParaReintento`, `debeGestionarErrorDePersistenciaYRecuperarseEnReintentoJms` y `debeRecuperarseDeFalloTemporalDePublicacionYCompletarElFlujoEnElSiguienteCiclo`	ter 2 y rec
8	**Manejo de mensajes inválidos y fallos reiterados (DLQ)**	**Unitaria:** `debeRechazarMensajesNulosOInvalidosYPropagarExcepcionParaDlq` **Integración:** `debeFallarAnteMensajeInvalidoYEnrutarloADlqTrasReintentos` y `debeVerificarPoliticaDeReintentosAnteFallosReiterados`	**Ev** (Co mo me en
9	**Generación de los esquemas y tablas requeridos (`grupoasd_dev` y `grupoasd_prod`)**	**Integración (PostgreSQL 16 real):** `debeGenerarEsquemaGrupoAsdDevYTablaProducto` y `debeGenerarEsquemaGrupoAsdProdYTablaProducto`	**Ev y 9** mo esc

5.2. Suite de 33 Pruebas Automatizadas: 15 Unitarias + 15 de Integración + 3 Flujo Completo E2E (`mvn clean verify`)

- **15 Pruebas Unitarias Puras (JUnit 5 + Mockito `@ExtendWith(MockitoExtension.class)`):**
  - En `ProductoServiceUnitTest` (Componente 1 — 9 tests): registro válido, normalización de código, rechazo de duplicados, rechazo de datos inválidos, construcción de mensaje JMS, identificación de pendientes por el Job, manejo de fallo de publicación, confirmación válida e idempotencia ante confirmaciones inválidas/repetidas.
  - En `ProductoSincronizacionServiceUnitTest` (Componente 2 — 6 tests): persistencia de nuevo producto, prevención de duplicados, manejo de concurrencia (`DataIntegrityViolationException`), construcción de confirmación, no confirmación ante fallo de BD y rechazo de mensajes inválidos.
- **15 Pruebas de Integración Reales (`@SpringBootTest` + `MockMvc` + PostgreSQL 16 `grupoasd_test_db` + ActiveMQ 6.3.2):**

    - En `ProductPublisherIntegrationTest` (Componente 1 — 9 tests) y `ProductConsumerIntegrationTest` (Componente 2 — 6 tests, incluyendo `ActiveMQ.DLQ` y `Awaitility`).
- **3 Pruebas de Flujo Completo entre Ambas Aplicaciones (`FlujoCompletoSincronizacionE2ETest`):**
    - Ciclo completo de 10 pasos (`grupoasd_dev.producto` → ActiveMQ → `grupoasd_prod.producto` → confirmación JMS → `notificado = true`), idempotencia extremo a extremo ante mensajes repetidos y recuperación ante fallo temporal de publicación.

```
[INFO] Reactor Summary for Grupo ASD - Product Synchronization System 1.0.0:

[INFO]

[INFO] Grupo ASD - Product Synchronization System ......... SUCCESS [  0.176 s]

[INFO] Product Publisher Service (Componente 1 - grupoasd_dev) SUCCESS [ 59.642 s] (21 tests:
9 unit + 9 integration + 3 E2E)

[INFO] Product Consumer Service (Componente 2 - grupoasd_prod) SUCCESS [ 25.004 s] (12 tests:
6 unit + 6 integration)

[INFO] ------------------------------------------------------------------------

[INFO] BUILD SUCCESS — Total: 33 tests run (15 Unitarias + 15 Integración + 3 Flujo Completo
E2E), 0 failures, 0 errors

[INFO] ------------------------------------------------------------------------
```

5.3. Evidencias Visuales Detalladas por Cada uno de los 9 Escenarios de Prueba

Escenario 1 (Evidencia 1A): Registro exitoso de un producto mediante la API REST (`POST /api/v1/productos` → `201 Created`)

Qué se probó: Envío de petición `POST http://localhost:8080/api/v1/productos` en Postman con los datos del producto `PROD-001`. El servicio valida la solicitud, guarda en `grupoasd_dev.producto` y responde HTTP **`201 Created`** con `"publicado": false` y `"notificado": false`.

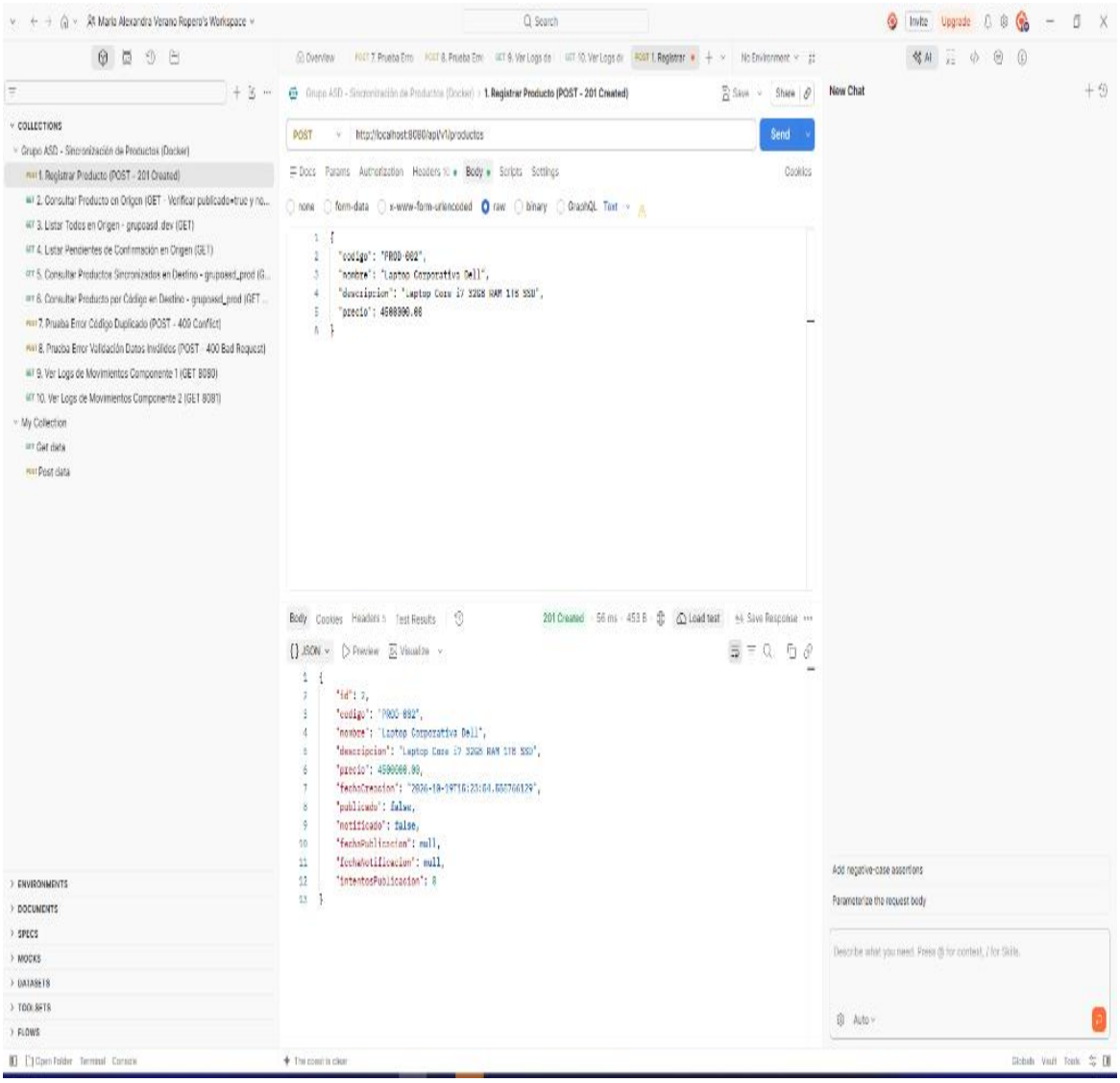


Figura 1A: Registro exitoso en Postman devolviendo 201 Created con publicado=false y notificado=false.

Escenario 1 y 4 (Evidencia 1B): Publicación automática por el Job y confirmación desde Destino (`publicado: true` y `notificado: true`)

Qué se probó: Al consultar `GET http://localhost:8080/api/v1/productos/PROD-001` tras ejecutarse el ciclo del Job (10 segundos), se comprueba que el producto fue publicado en ActiveMQ (`"publicado": true`) y confirmado por el Componente 2 (**`"notificado": true`**).

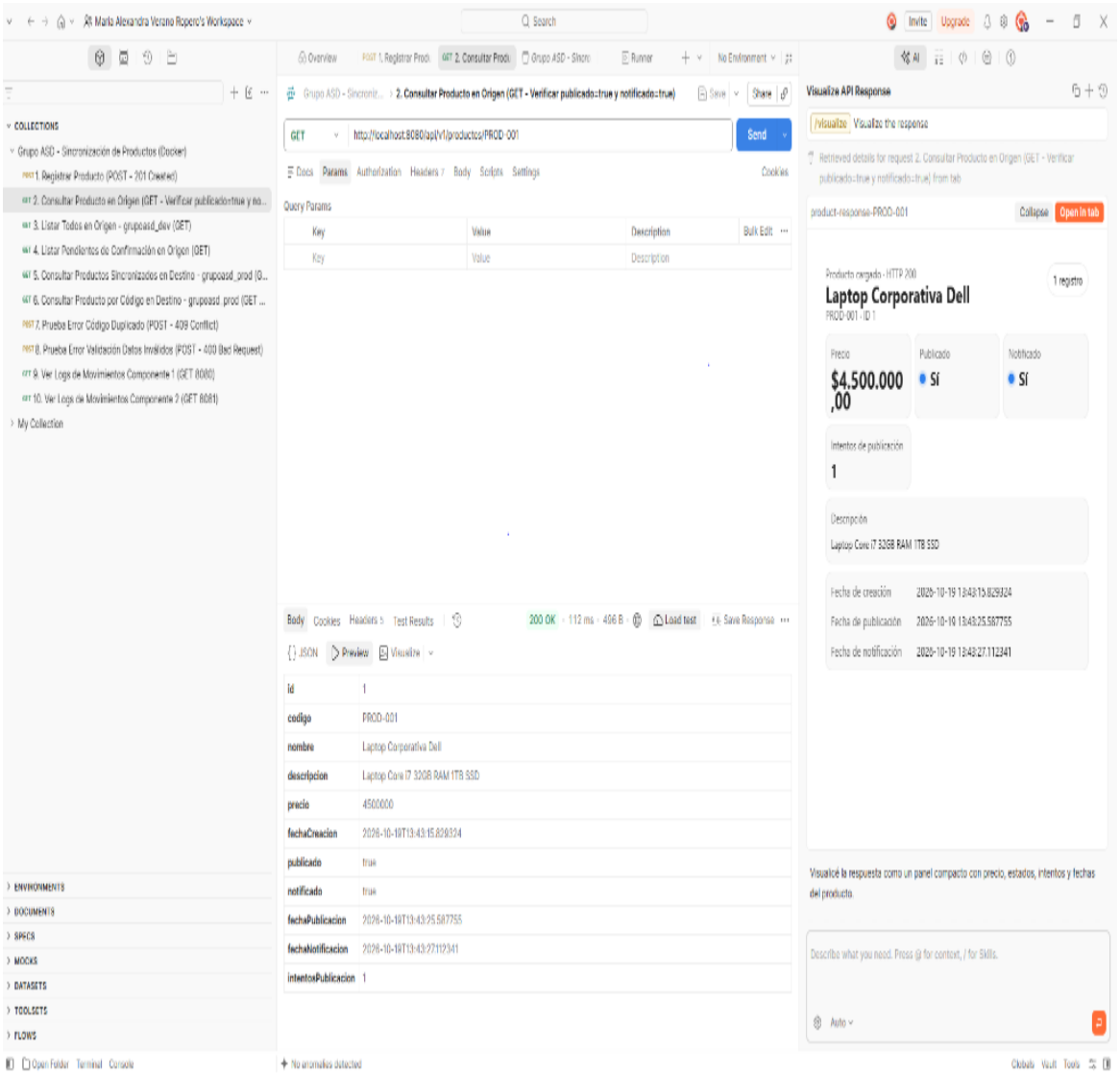


Figura 1B: Consulta GET en Postman evidenciando el paso automático a publicado=true y notificado=true.

Escenario 2 (Evidencia 2A): Rechazo de productos con código duplicado (`POST /api/v1/productos` → `409 Conflict`)

Qué se probó: Se intentó registrar por segunda vez el código `PROD-001`. El servicio rechaza la operación con HTTP **`409 Conflict`** ("*Ya existe un producto registrado con el código 'PROD-001'*") evitando duplicados en origen.

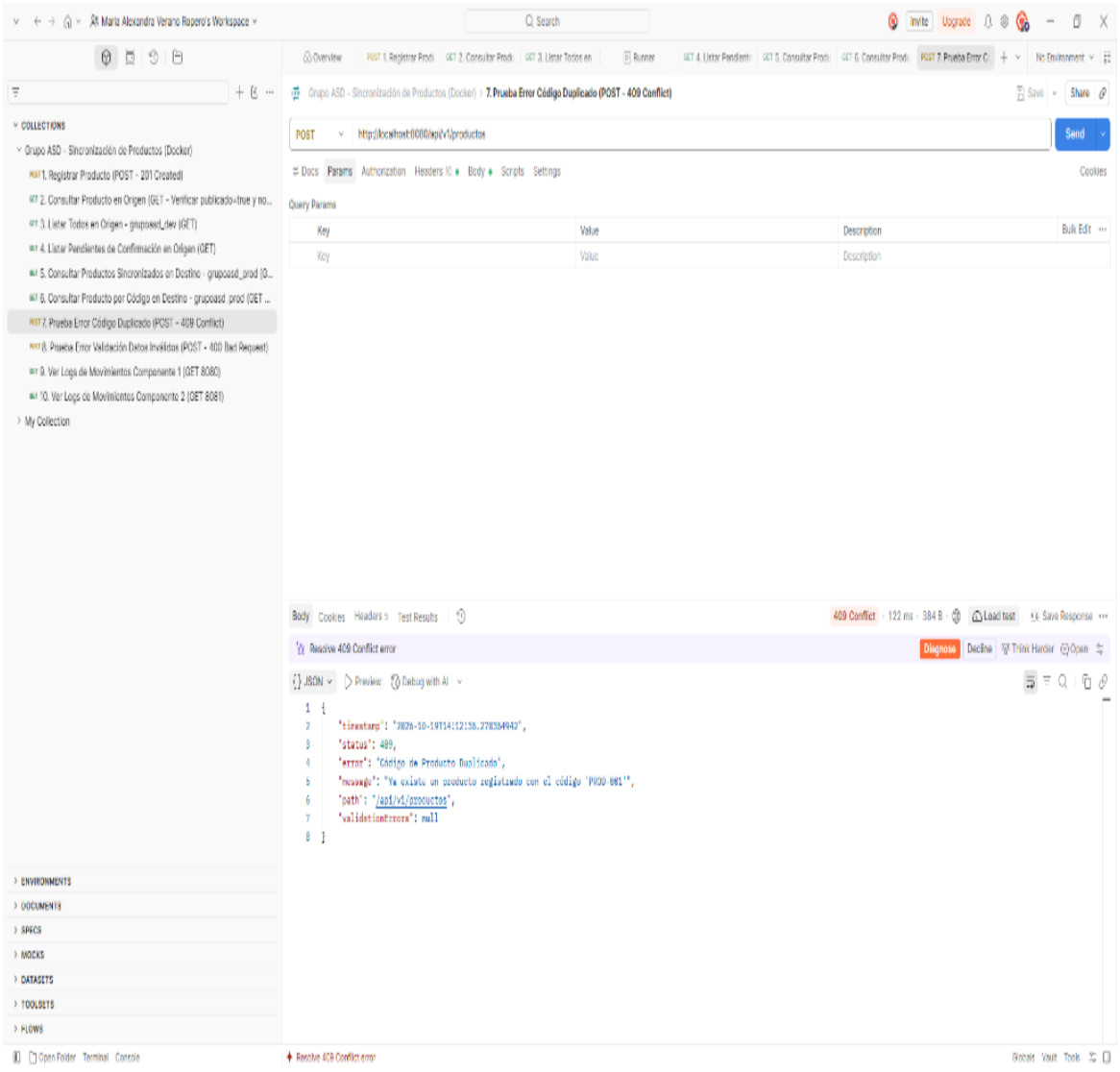


Figura 2A: Rechazo HTTP 409 Conflict en Postman ante código de producto duplicado.

Escenario 2 (Evidencia 2B): Rechazo de productos con información inválida (`POST /api/v1/productos` → `400 Bad Request`)

Qué se probó: Se envió un JSON con código vacío, nombre/descripción demasiado cortos y precio negativo (`-500`). El servicio responde HTTP **`400 Bad Request`** detallando los errores de validación de cada campo.

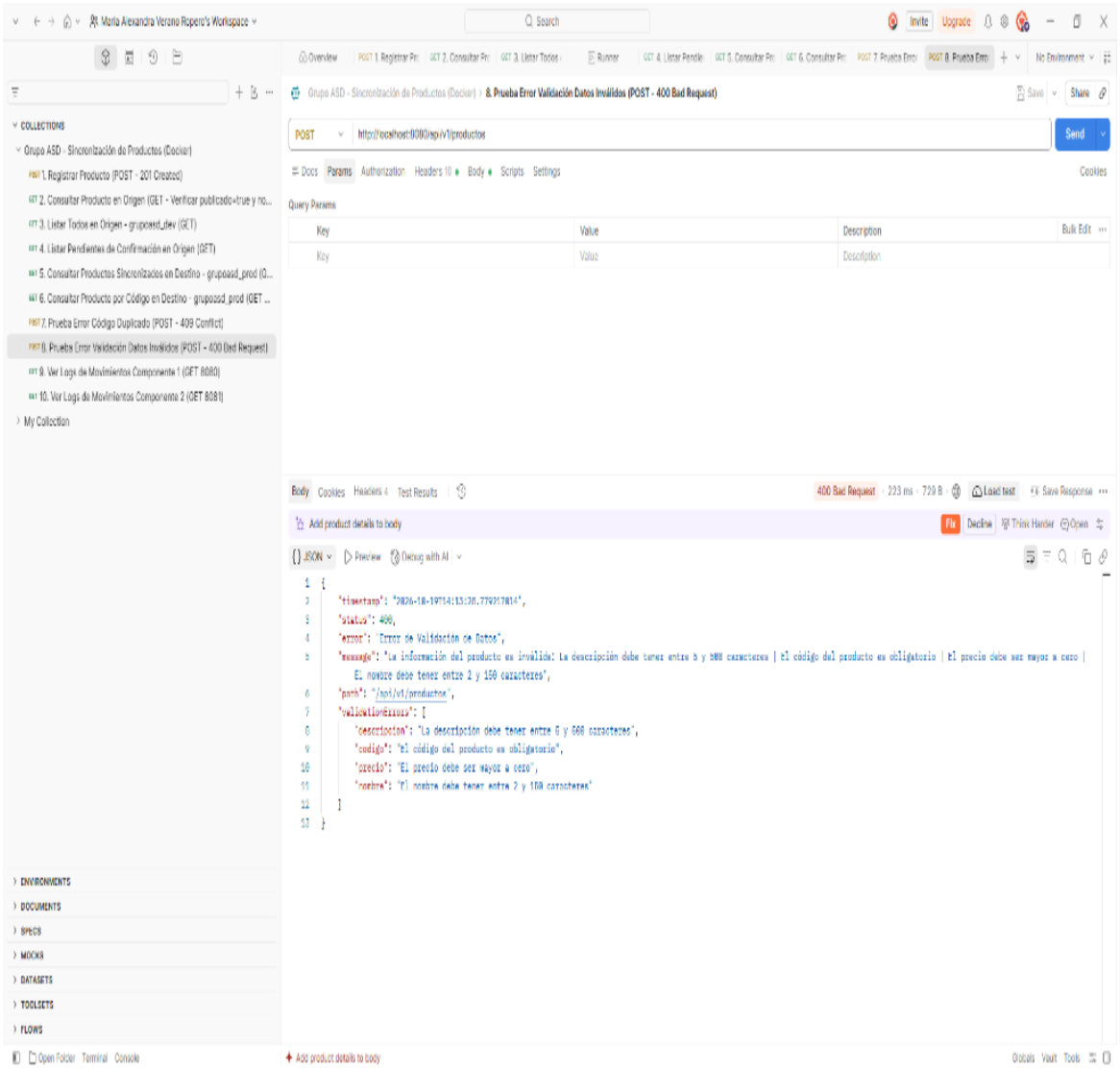


Figura 2B: Rechazo HTTP 400 Bad Request en Postman por datos inválidos.

Escenario 3 (Evidencia 3A y 3B): Identificación de productos pendientes por parte del Job

Qué se probó: Se consultó `GET /api/v1/productos/pendientes-publicacion` comprobando que queda en `[]` (0 pendientes) tras el ciclo del Job, y en `GET /api/v1/productos/logs-movimientos` se verificó el log con fecha y hora donde el Job identifica y publica los productos pendientes.

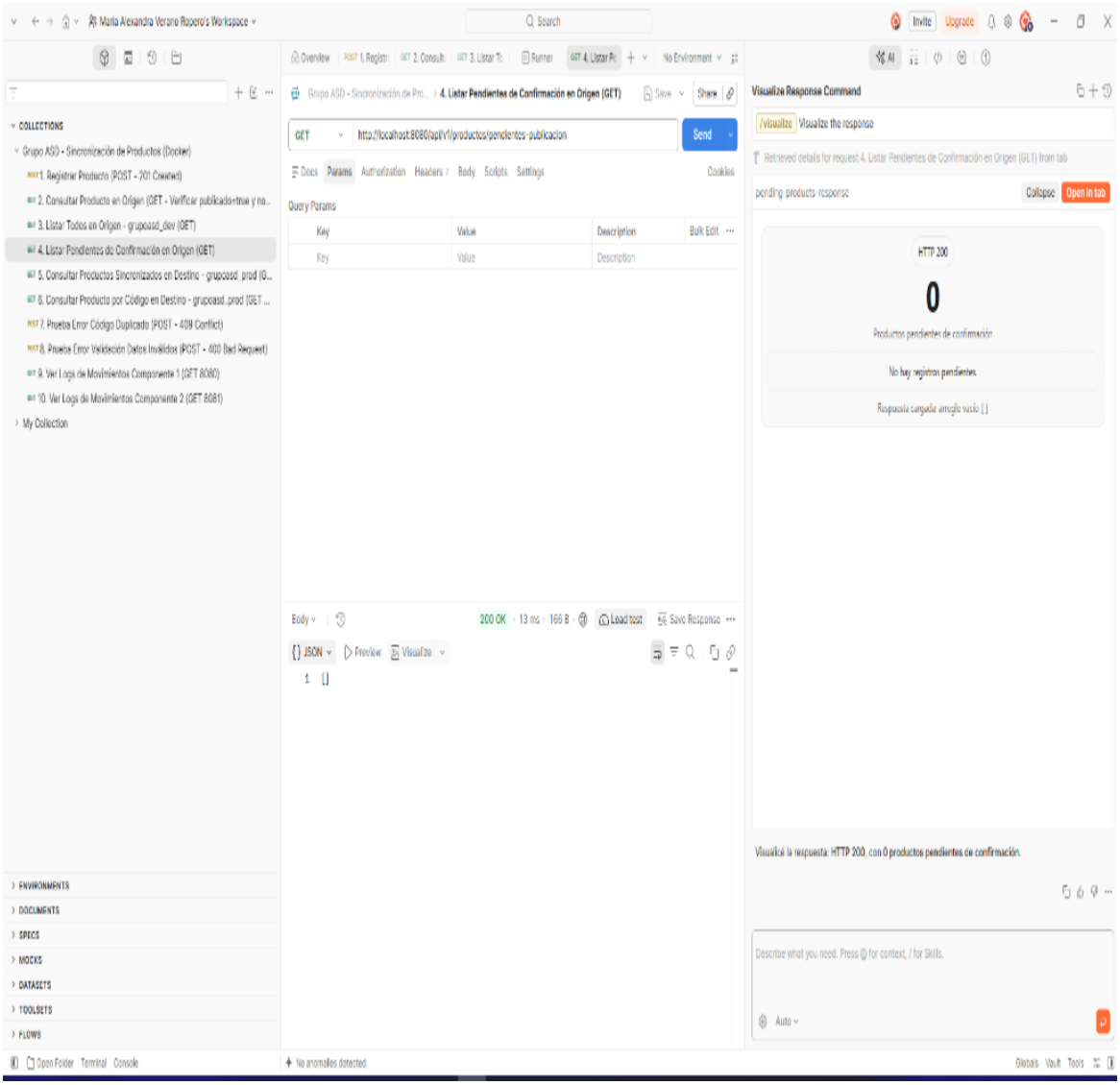


Figura 3A: Respuesta [] (0 productos pendientes) tras completarse el ciclo automático del Job.



Figura 3B: Bitácora de logs del Componente 1 mostrando la identificación de productos pendientes por el Job.

Escenario 4 (Evidencia 4): Publicación correcta de los mensajes en ActiveMQ (Consola Web Puerto 8161)

Qué se probó: En la consola administrativa de ActiveMQ (`http://localhost:8161/admin/queues.jsp`) se comprueba la publicación y consumo en `grupoasd.productos.registro.queue` y `grupoasd.productos.notificacion.queue` con consumidores activos y 0 mensajes represados.

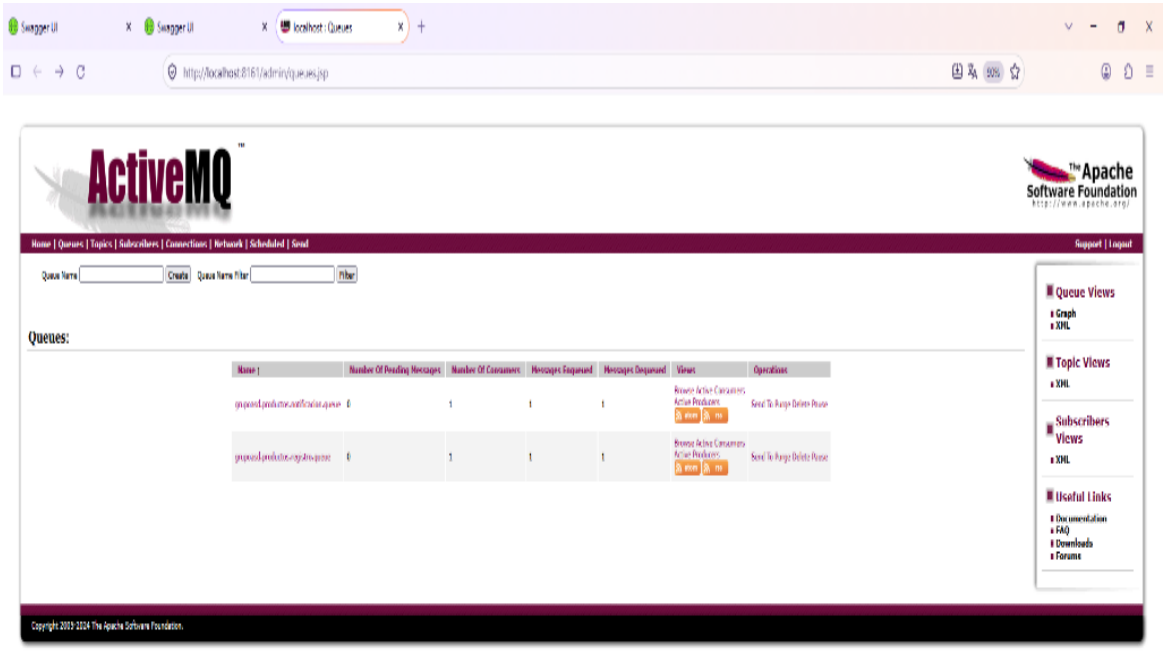


Figura 4: Consola Web de ActiveMQ verificando la publicación y consumo en ambas colas.

Escenario 5 (Evidencia 5A y 5B): Consumo y persistencia de productos en `grupoasd_prod` (Puerto 8081)

Qué se probó: Se consultó `GET http://localhost:8081/api/v1/productos-destino` y `GET http://localhost:8081/api/v1/productos-destino/logs-movimientos` en el Componente 2, comprobando que el mensaje fue recibido desde ActiveMQ, validado, persistido en `grupoasd_prod.producto` y confirmado con ACK.

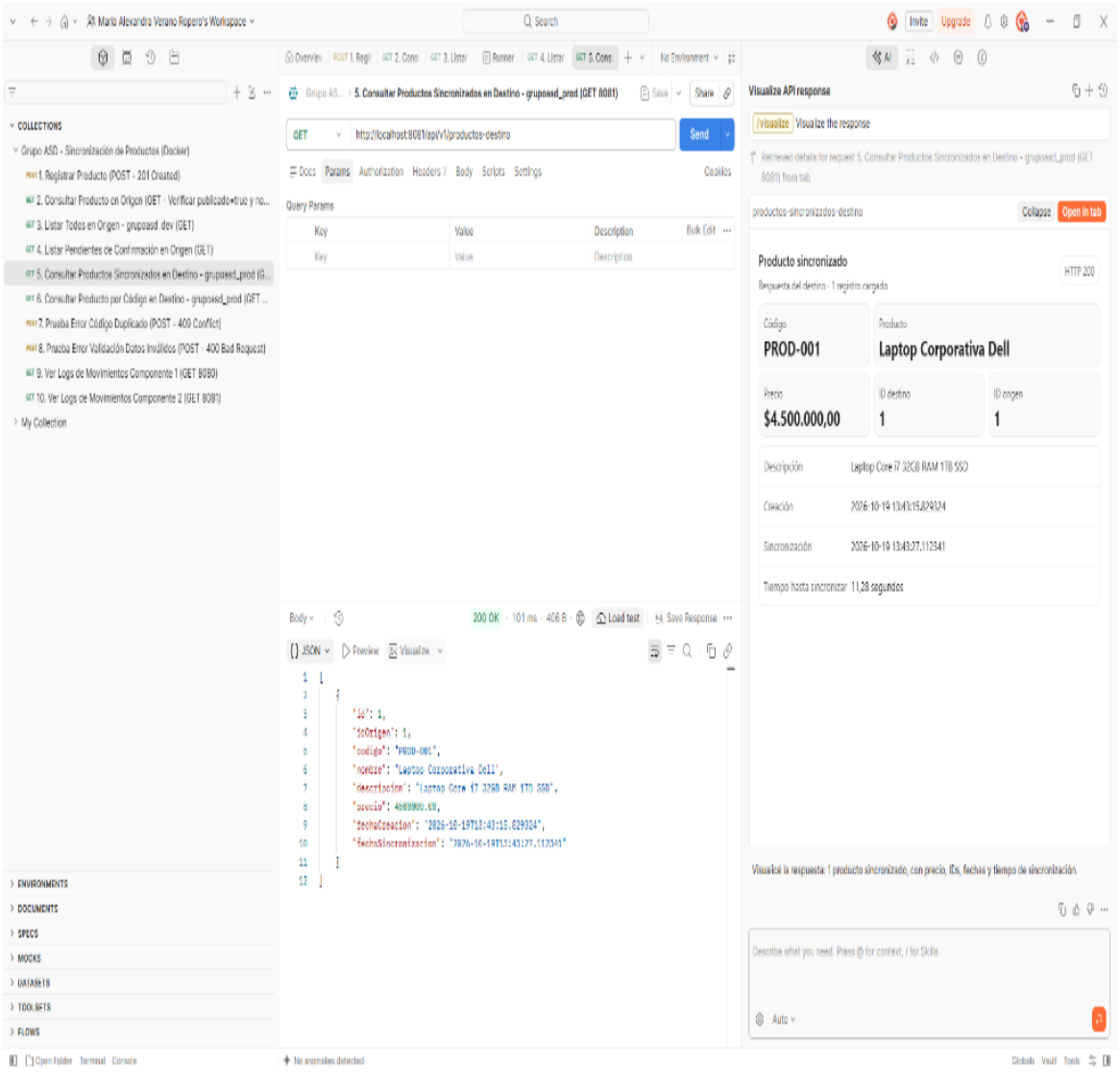


Figura 5A: Consulta en el Componente 2 (puerto 8081) comprobando el producto persistido en grupoasd_prod.

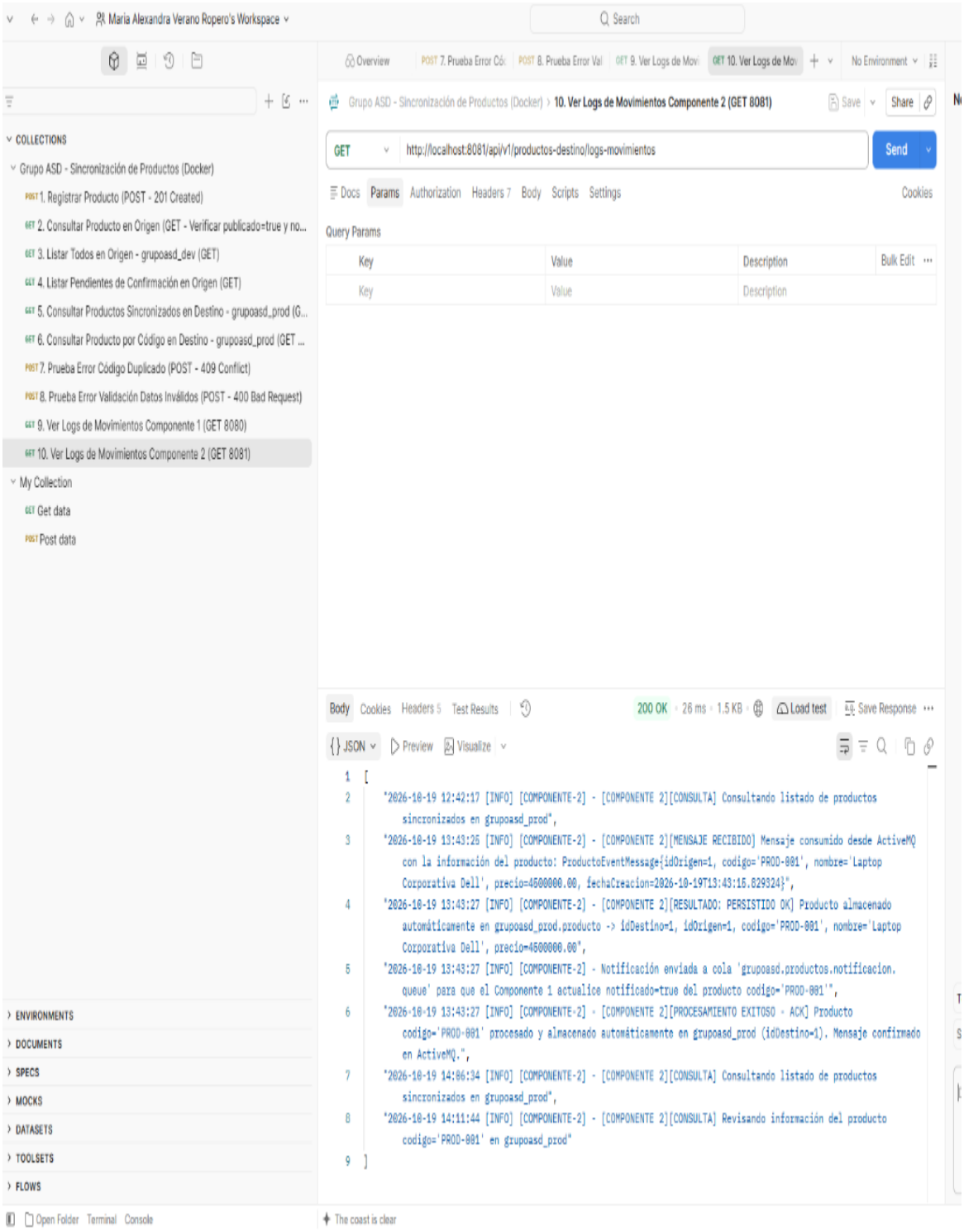


Figura 5B: Bitácora de logs del Componente 2 mostrando recepción JMS, persistencia y confirmación ACK.

Escenario 7 (Evidencia 7): Recuperación ante fallos temporales de publicación o persistencia

Qué se probó: Se detuvo temporalmente el contenedor del Componente 2 y se registró `PROD-002`. En `GET /api/v1/productos/pendientes-publicacion` se observa en estado `"publicado": true`

y `"notificado": false` durante la caída. Al iniciar nuevamente el Componente 2, el producto se sincroniza y pasa automáticamente a `notificado = true`.

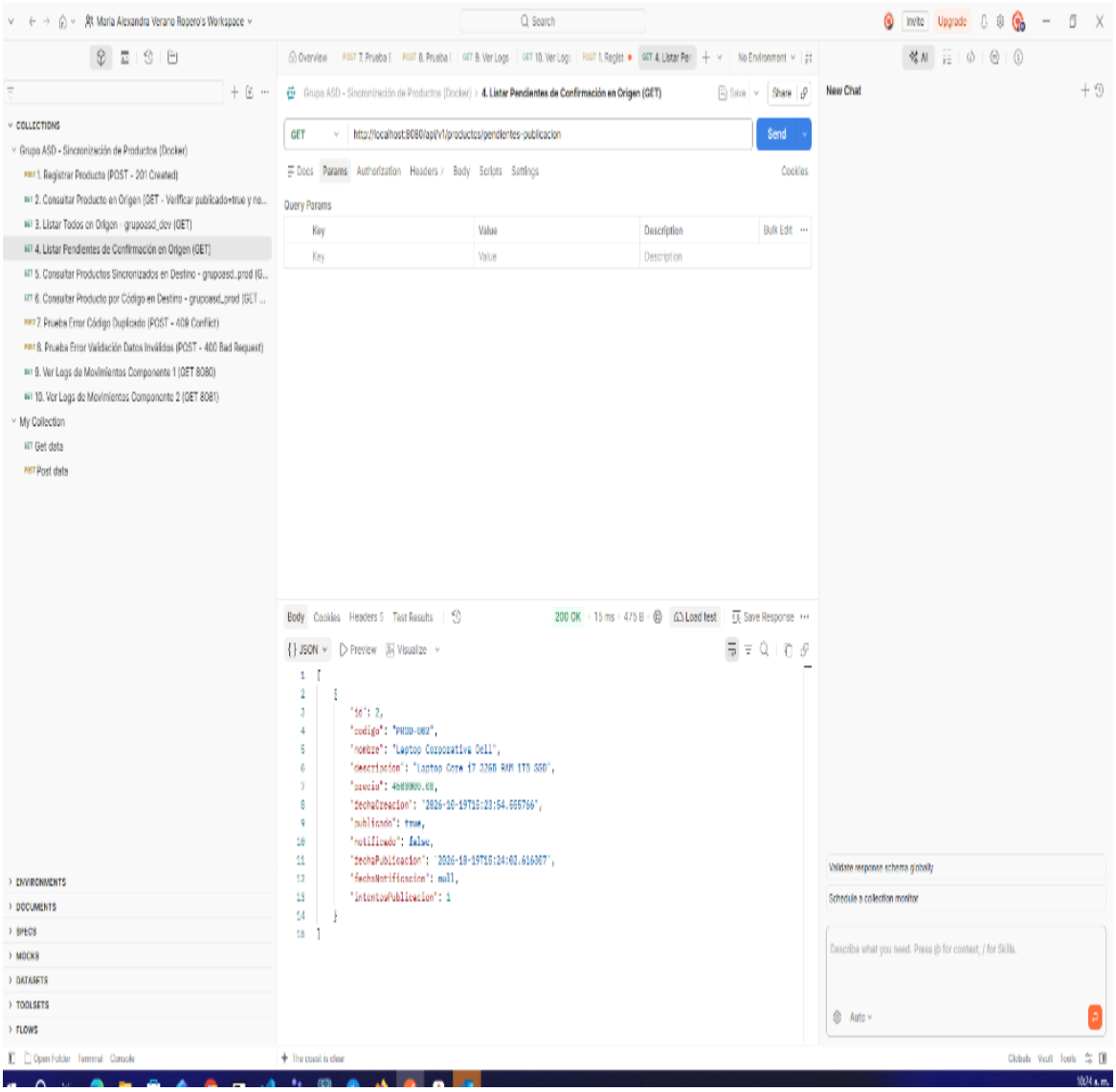


Figura 7: Producto PROD-002 en estado controlado (publicado=true, notificado=false) durante caída temporal del Componente 2.

Escenario 6 y Escenario 8 (Evidencia 8): Prevención de duplicados ante recepción repetida y manejo de mensajes inválidos en `ActiveMQ.DLQ`

Qué se probó: (1) Se procesaron 10 mensajes en `grupoasd.productos.registro.queue` (incluyendo reenvíos del mismo producto) sin generar duplicados en `grupoasd_prod.producto`; y (2) al enviar un mensaje inválido, tras agotar los 3 reintentos exponenciales sin confirmarlo, ActiveMQ lo transfirió automáticamente a **`ActiveMQ.DLQ` (`Number Of Pending Messages: 1`)**.

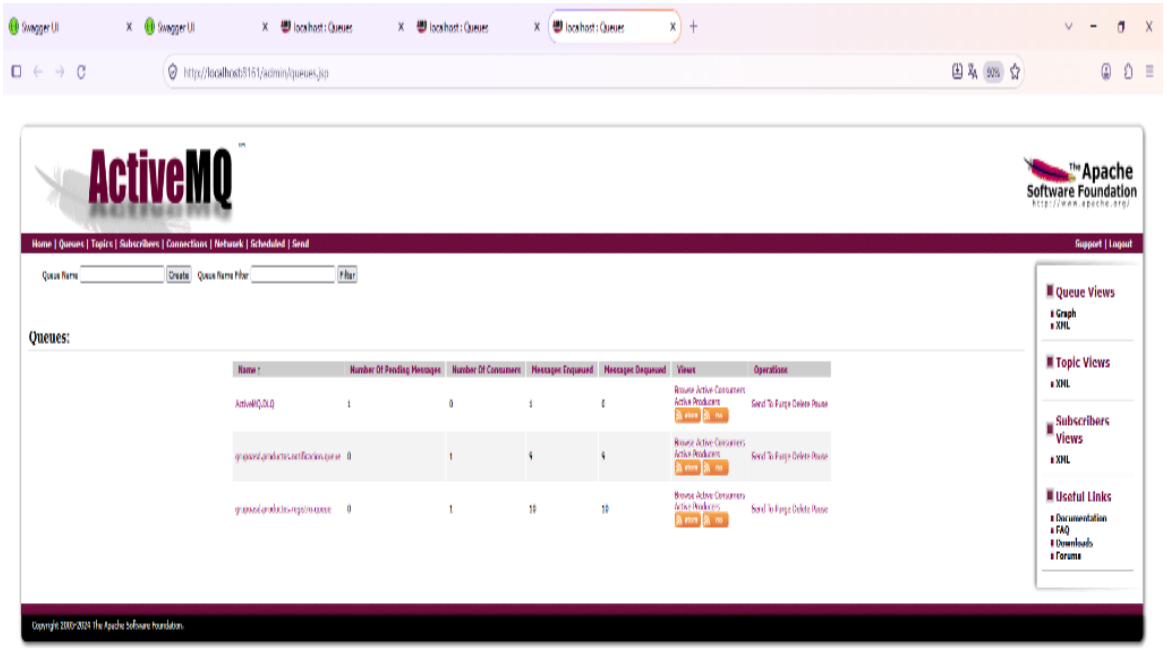


Figura 8: Consola de ActiveMQ evidenciando los 10 mensajes procesados sin duplicar y el mensaje inválido en ActiveMQ.DLQ.

Escenario 9 (Evidencias 9A, 9B y 9C): Generación automática de los esquemas `grupoasd_dev` y `grupoasd_prod` y tablas `producto` en PostgreSQL (pgAdmin 4)

Qué se probó: Conectado desde **pgAdmin 4** al contenedor PostgreSQL de Docker (base de datos `grupoasd_db`), se verificó la creación de los esquemas `grupoasd_dev` y `grupoasd_prod`, sus tablas `producto` y los registros sincronizados en ambas tablas.

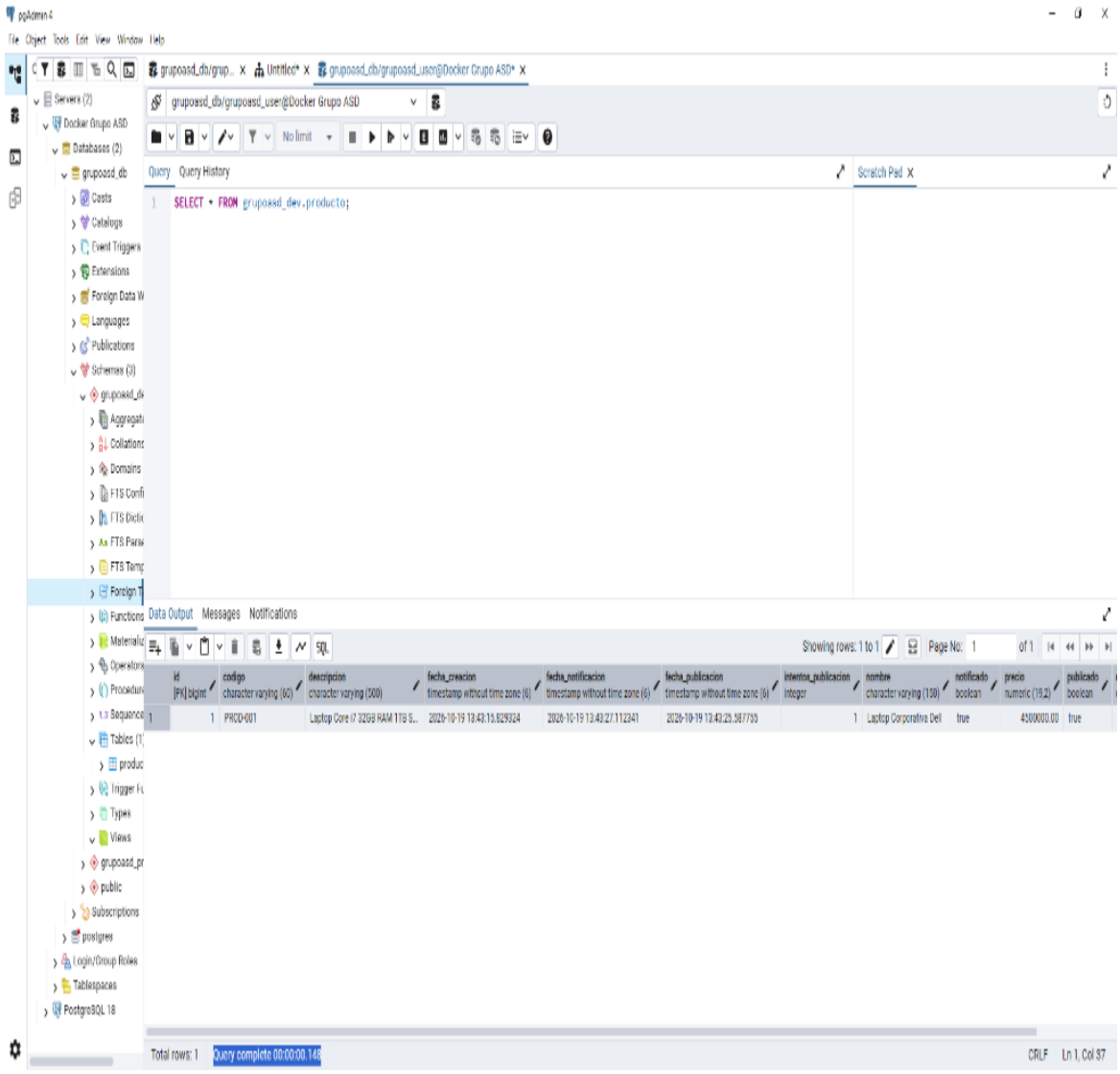


Figura 9A: Esquemas grupoasd_dev y grupoasd_prod en pgAdmin 4 y consulta SELECT sobre grupoasd_dev.producto.

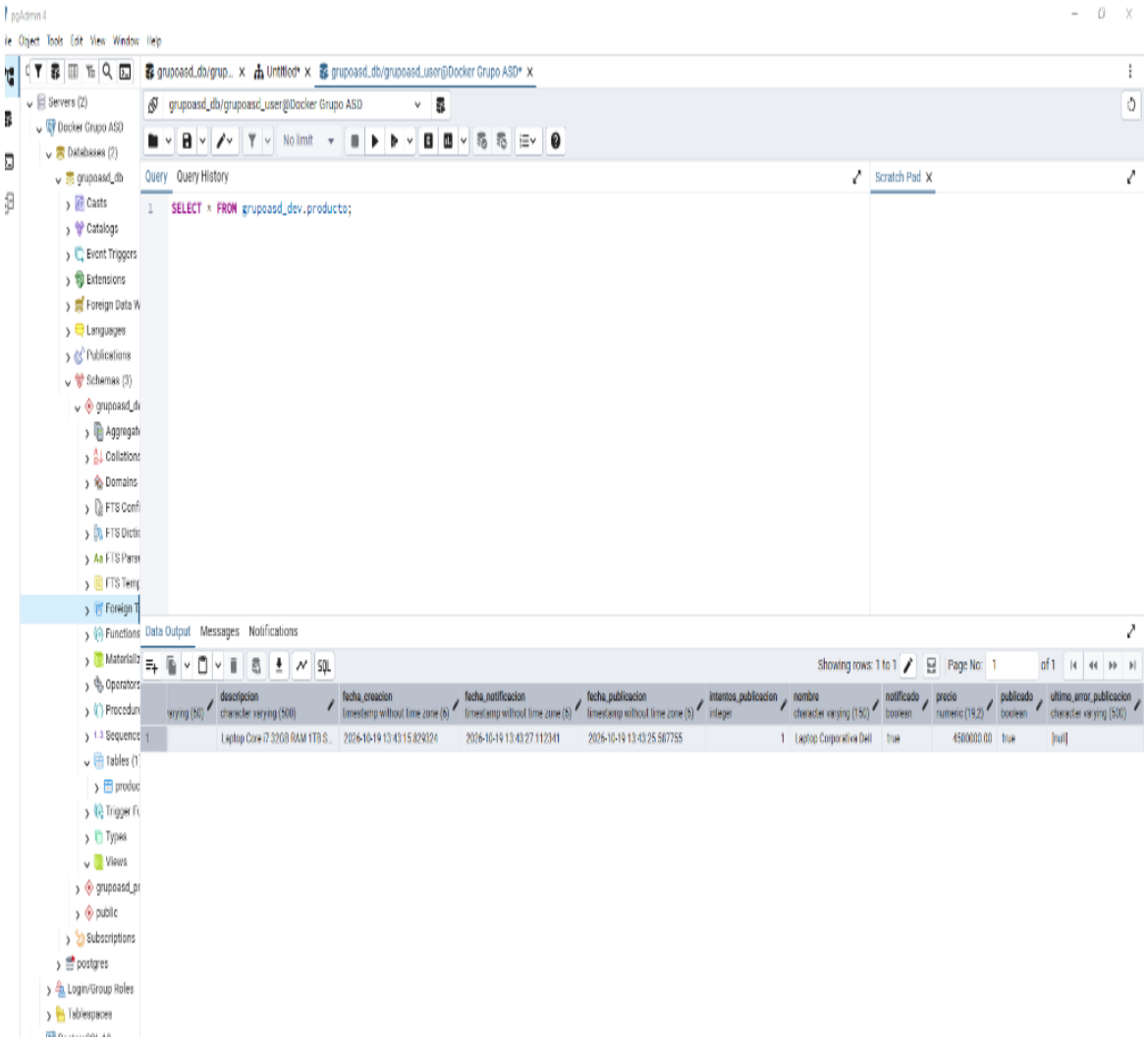


Figura 9B: Detalle de columnas fecha_publicacion, fecha_notificacion, intentos_publicacion y publicado/notificado en grupoasd_dev.producto.

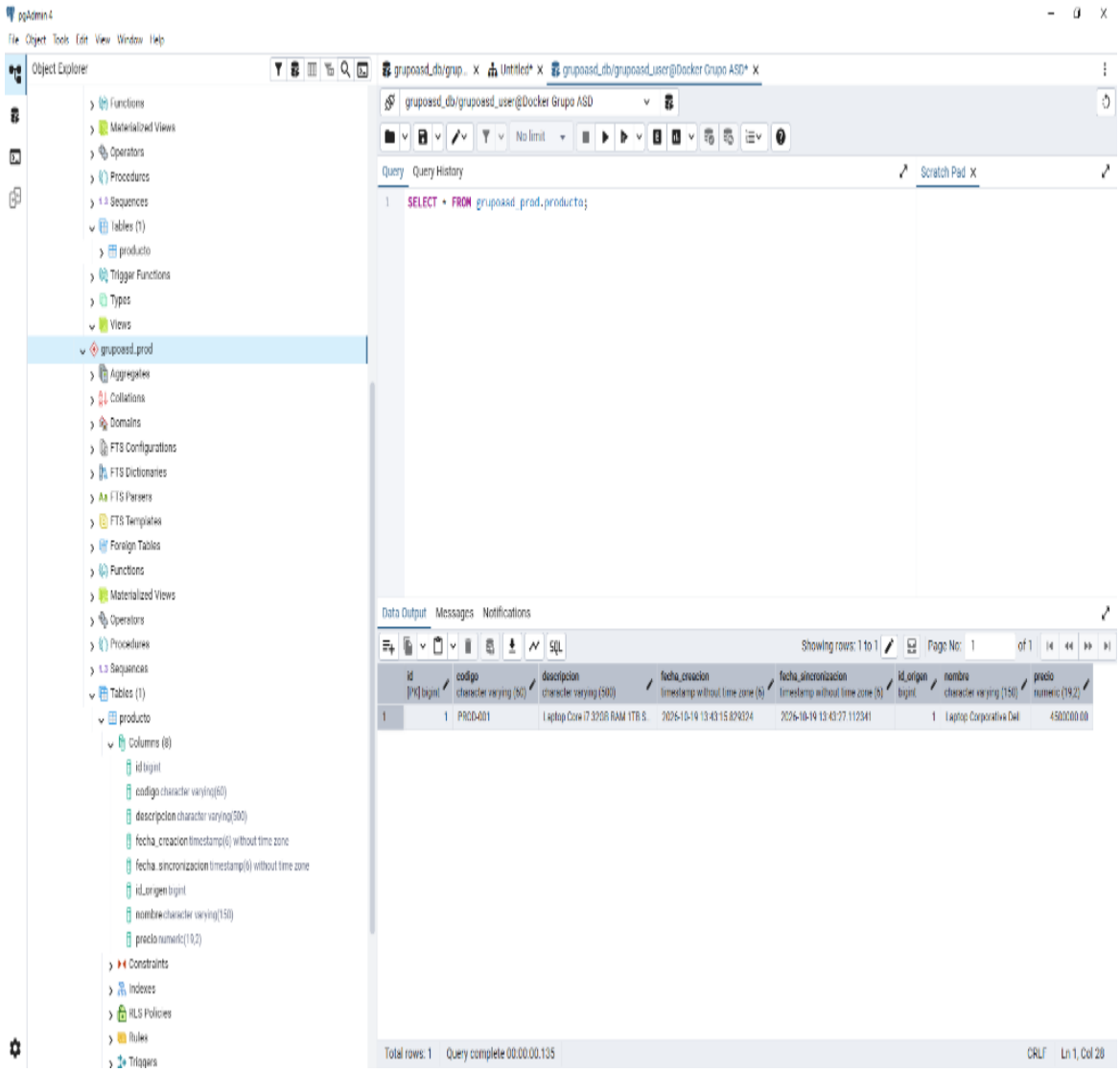


*Figura 9C: Consulta SELECT * FROM grupoasd_prod.producto en pgAdmin 4 comprobando el esquema y tabla de producción.*

ENTREGABLE 6: Breve Explicación del Mecanismo de Identificación de Productos Pendientes, Prevención de Duplicados y Manejo de Errores

6.1. Mecanismo de Identificación de Productos Pendientes

- Cada producto registrado por la API REST se almacena en `grupoasd_dev.producto` con dos banderas booleanas de control: **`publicado = false`** (pendiente de enviar a ActiveMQ) y **`notificado = false`** (pendiente de confirmación desde `grupoasd_prod`).
- El Job programado (`ProductoPublicacionJob`) se ejecuta periódicamente según el intervalo configurado en `application.yml` (`fixed-delay-ms: 10000`) y consulta los

registros con `publicado = false` ordenados por fecha de creación ascendente (FIFO) apoyándose en el índice compuesto `idx_producto_dev_publicado_fecha (publicado, fecha_creacion)`.

- Una vez publicado el mensaje JSON en ActiveMQ, el registro cambia a `publicado = true` y registra `fecha_publicacion`. Cuando el Componente 2 confirma su persistencia en producción, cambia a `notificado = true` y registra `fecha_notificacion`.
- Si un producto lleva más de 20 segundos con `publicado = true` pero `notificado = false`, el Job lo reenvía automáticamente para garantizar convergencia eventual.

6.2. Mecanismo de Prevención de Duplicados

- **En el sistema de origen (`grupoasd_dev`):** Antes de insertar se normaliza el código en mayúsculas y se valida con `existsByCodigoIgnoreCase(codigo)`; si ya existe, se rechaza con HTTP `409 Conflict`, respaldado por la restricción `UNIQUE (codigo)` en base de datos.
- **En el sistema de destino (`grupoasd_prod`):** El consumidor JMS implementa el patrón **Idempotent Consumer** en `ProductoSincronizacionServiceImpl`: antes de insertar, consulta `findByCodigoIgnoreCase(codigo)`. Si el producto ya existe en `grupoasd_prod.producto`, omite el `INSERT`, deja constancia en el log (`[RESULTADO: DUPLICADO OMITIDO]`) y reenvía la confirmación al Componente 1. Adicionalmente, la tabla tiene restricción `uk_producto_prod_codigo UNIQUE (codigo)` y captura de `DataIntegrityViolationException` ante concurrencia.

6.3. Mecanismo de Manejo de Errores, Reintentos y Dead Letter Queue (`ActiveMQ.DLQ`)

- **Fallos de publicación en el Componente 1:** Cada producto se procesa en una transacción independiente (`Propagation.REQUIRES_NEW`). Si ActiveMQ no está disponible, se captura la excepción, el producto conserva `publicado = false`, incrementa `intentos_publicacion` y almacena el detalle en `ultimo_error_publicacion` para reintentarse en el siguiente ciclo del Job.
- **Fallos de validación o persistencia en el Componente 2:** El listener JMS opera con sesión transaccional **`Session.SESSION_TRANSACTED`**. Si un mensaje tiene datos inválidos o ocurre un fallo de conexión/persistencia, se lanza una excepción que hace rollback de la sesión JMS, garantizando que **ningún mensaje se confirme como exitoso si el producto no fue persistido correctamente**.
- **Estrategia de reintentos y DLQ:** ActiveMQ aplica la `RedeliveryPolicy` configurada en `JmsConsumerConfig` con **backoff exponencial** (`maximumRedeliveries = 3`, retardo inicial `2000 ms`, multiplicador `2.0`: 2s → 4s → 8s). Si tras agotar los 3 reintentos el mensaje continúa fallando, ActiveMQ lo transfiere automáticamente a la cola de mensajes muertos **`ActiveMQ.DLQ`**.

ENTREGABLE 7 (Valor Agregado): Ejecución del Entorno Completo mediante Docker Compose

En la raíz del proyecto se entrega el archivo `docker-compose.yml` que orquesta de forma automática los **4 contenedores** sobre una red dedicada (`grupoasd-network`) con verificaciones de salud (`healthcheck`) para garantizar el orden de arranque:

Contenedor Docker	Imagen / Dockerfile	Puertos Expuestos	Rol y Configuración Automática
`grupoasd-postgres`	`postgres:16-alpine`	`5432:5432`	Crea automáticamente la base de datos `grupoasd_db` y ejecuta `init-schemas.sql` creando `grupoasd_dev` y `grupoasd_prod`.
`grupoasd-activemq`	`docker/activemq/Dockerfile` (Java 25 + ActiveMQ Classic)	`61616:61616` `8161:8161`	Bróker JMS OpenWire y Consola Web administrativa con soporte de colas de registro, confirmación y `ActiveMQ.DLQ`.
`grupoasd-publisher-service`	`product-publisher-service/Dockerfile` (Multi-stage Java 25 LTS)	`8080:8080`	Inicia cuando PostgreSQL y ActiveMQ están en estado `healthy`; genera la tabla `grupoasd_dev.producto` y activa el Job.
`grupoasd-consumer-service`	`product-consumer-service/Dockerfile` (Multi-stage Java 25 LTS)	`8081:8081`	Inicia cuando PostgreSQL y ActiveMQ están en estado `healthy`; genera la tabla `grupoasd_prod.producto` y suscribe el consumidor JMS.